Claude Code

橫縱分析法深度研究報告

研究時間：2026-04-20 | 所屬領域：AI 軟體工程、開發者工具 | 研究對象類型：產品（CLI agent）| 作者：GenAI Frontiers

作者: GenAI Frontiers

> 研究時間：2026-04-20 | 所屬領域：AI 軟體工程、開發者工具 | 研究對象類型：產品（CLI agent）| 作者：GenAI Frontiers

一、一句話定義

Claude Code 是 Anthropic 2025 年 2 月 24 日從 research preview 公開、5 月 22 日轉 GA 的「終端原生 AI 編碼 agent」——它不是 chatbot，不是 IDE 擴充，而是一個把 LLM + 工具迴圈 + 權限系統 + 擴展機制包裝進 `claude` 這條指令裡的完整 agent runtime，截至 2026 年 4 月已貢獻 Anthropic 一半以上企業營收、佔公開 GitHub commits 的 4%，並在 Sonnet 4.5、Opus 4.6、Opus 4.7 三代模型的迭代中，把 AI 編碼的主流形態從「輔助補全」一路推到「數十小時無人值守的自主任務」。

二、縱向分析：從誕生到當下

2.1 史前時代：Anthropic 的遲疑與外部壓力（2024 上半）

要理解 Claude Code 為什麼會出現，得先看 2024 上半年的格局。

Claude 3 在 2024 年 3 月發布 Opus / Sonnet / Haiku 三個尺寸，Claude 3.5 Sonnet 在 6 月接著登場，coding 能力第一次明顯超過當時的 GPT-4。但此時 Anthropic 並沒有一個自家的編碼 agent 產品。真正把 Claude 帶進工程師日常的，是第三方——Cursor 在這個時段靠 Claude 3.5 Sonnet 打爆增長曲線，從不到一萬用戶推到數十萬；Windsurf、Continue、Cline 是以「支援 Claude」作為重要賣點在成長；Aider 這種更古典的 CLI 工具也全靠 Claude 的 API 在跑。

這是一個奇妙的錯位。Anthropic 手上握著 coding 能力最強的模型，但「Claude 怎麼寫程式」這個問題的答案，幾乎都由第三方來回答。2024 年中，Anthropic 內部大概沒有正式的戰略文件寫「我們要做自己的 coding tool」，但至少有兩件事讓這個方向變得不可迴避：

第一，他們看見 Cursor 的黏著曲線。一個用 Claude API 的外部工具能在幾個月內做到每月 $10M 以上的 ARR，而且大部分營收來自「用戶愛 Claude Sonnet」——這等於證明 coding 是 Claude 的結構性優勢，但這個優勢的變現管道完全不在自己手上。

第二，他們意識到 Claude 3.5 Sonnet 的 tool use 能力遠超行業平均，這代表一個「完全由 Claude 自主驅動」的 agent 是可行的，只要有人願意花時間把 infrastructure 搭起來。

所以當 Boris Cherny 在 2024 年 9 月加入 Anthropic 的時候，土壤已經鋪好了。Cherny 是 Meta 的前 principal engineer、《Programming TypeScript》的作者，以挑戰性難題為樂。他加入 Anthropic 的第一件事，用他自己的話說，是「熟悉一下 API」——他寫了個小 CLI，給 Claude 3.6（也就是後來對外命名為 Sonnet 3.5 v2 的版本）接上 filesystem 權限，問它「工程師現在都在聽什麼音樂」。這個 prototype 在 Anthropic 內部瘋傳。

Cherny 在 Pragmatic Engineer 的訪談裡承認，他並沒有預料到這會成為 Anthropic 最大的產品線之一。「我只是想寫個好玩的東西。」但這個「好玩的東西」揭示了一件他可能當時沒完全意識到的事——**Claude 的 tool use 能力已經強到你不需要複雜的 agent framework，只要給它幾個基本工具加上足夠長的迴圈，它就能自己把事情做完。**

這個發現，就是 Claude Code 的原點。

2.2 誕生：週末 hack 變成產品線（2024 Q4）

2024 年 11 月是個關鍵月份。

Anthropic 內部發布了第一個可以被稱為「Claude Code」的 dogfooding 版本，距離 Cherny 加入大約兩個月。根據 Pragmatic Engineer 的報導，Day 1 就有 20% 的 Anthropic 工程團隊開始用，Day 5 這個數字是 50%。Sid Bidasaria（工程師 #2，前 Robinhood / Rubrik / Harness，後來成為 sub-agents 的主要設計者）在這個月加入。Cat Wu 擔任 founding PM。

同一個月，Anthropic 在 11 月 25 日公開發布了 **Model Context Protocol（MCP）**。MCP 這個協議最初看起來和 Claude Code 關係不大——它定義的是「AI 系統如何與外部資料來源溝通」，首發時帶的 reference servers 是 Google Drive、Slack、GitHub、Postgres 這些，看起來是要讓 Claude 桌面版 app 變得更能接外部資料。但如果你把 Claude Code 的發展軌跡往後看，會發現 MCP 其實是為 Claude Code 鋪路的。因為 Claude Code 從第一天起就沒打算做閉環——它要讓使用者把自己的工具接進來，而「怎麼接」這件事需要一個協議。MCP 就是那個協議。

換句話說，Anthropic 在 2024 年 11 月做的不是兩件事，而是一件事的兩個面向：一個是面向使用者的 agent runtime（Claude Code），另一個是這個 agent runtime 的外部擴展標準（MCP）。後者的商業意義後來被嚴重低估——MCP 不只讓 Claude Code 能接外部工具，它等於是讓 Anthropic 免費獲得了整個第三方生態為它蓋的基礎設施。

值得注意的是，Claude Code 的技術選型在這個階段就鎖定了：TypeScript + React + Ink（把 React 組件模型搬到終端機的 TUI 框架）+ Bun 相容。這些選擇看起來平淡無奇，但在後來的 v2.1.88 源碼洩漏事件裡我們會看到，這個技術棧讓 Claude Code 得以把 389 個 React 組件、104 個 hooks、895KB 的 REPL.tsx 全部塞進一個終端機——這種複雜度級別在 CLI 工具史上是沒有前例的。

2.3 Research Preview 時代：2 月的同日發布（2025 Q1）

2025 年 2 月 24 日是 Claude Code 公開歷史的第一天。

這一天 Anthropic 同時發布了三件事：Claude 3.7 Sonnet（他們第一個 hybrid reasoning 模型）、Extended Thinking 功能、以及 Claude Code（limited research preview）。三者的公開定位是配套的——3.7 Sonnet 是能做深度思考的模型，Extended Thinking 讓你在 API 層面啟用這個能力，Claude Code 則是「讓你在終端機裡直接體驗這能力」的載體。

官方在 blog post 裡給 Claude Code 的形容是：「an agentic coding tool that lives in your terminal, searches your codebase, edits files, writes tests, and commits to GitHub.」他們還給出一個 aggressive 的 benchmark 數字——「Claude Code 能在一次 pass 裡完成工程師原本要 45+ 分鐘的任務」。

但這個階段的 Claude Code 是**半封閉**的。它的形式是 npm package `@anthropic-ai/claude-code`，但需要從文檔 opt-in 拿取早期存取權。功能上它只有最基礎的 tool set（Read、Write、Edit、Bash、Grep、Glob），還沒有 hooks、沒有 sub-agents、沒有 skills、沒有 plugins、沒有 IDE 整合。要到 GA 為止，它的版本號停留在 v0.2.x 系列——從 v0.2.1 到 v0.2.125（5 月 21 日的最後一個 beta），連續快速迭代了三個月。

為什麼選這個時間點公開？一個合理的推論是：3.7 Sonnet 的 extended thinking 讓 agent 能連續做幾十個 tool call 而不失控，這是 Claude Code 從「有趣的 demo」變成「能真正做事」的關鍵門檻。沒有 extended thinking 之前，一個編碼 agent 跑到第 10 個 tool call 就容易迷失方向；有了之後，它可以一次跑 30-50 個 tool call 完成複雜任務。

這個時期 Anthropic 對 Claude Code 的商業期待，從事後看其實很保守。它走的是 API pay-per-token 計費——你在 terminal 裡打一個 prompt，背後扣你 API quota。沒有專屬訂閱、沒有特殊方案。一個合理的假設是：Anthropic 當時把 Claude Code 視為「讓開發者認識 Claude tool use 能力」的 demo，而不是主力營收產品。

這個假設在 4 月被第一次修正。

2.4 Max Plan：從 pay-per-token 到訂閱（2025-04-09）

2025 年 4 月 9 日，Anthropic 推出 Claude Max Plan——$100/月的 Max 5x 和 $200/月的 Max 20x。

表面上看這是 Claude.ai 這個 chat 產品的訂閱方案更新。但真正的信號藏在細節裡：Max 計劃的 usage quota 不只涵蓋 Claude.ai 網頁版，還涵蓋 Claude Code。也就是說，從這天起，你可以用 $100 或 $200 的固定月費來換取在 Claude Code 裡「近似無限」的使用量，不用再對每個 prompt 擔心 token 成本。

這是個細微但致命的轉變。對一個會連續跑幾小時 agent loop 的產品，pay-per-token 的計費模式天然是反增長的——使用得越多，成本越感人。Max Plan 把這個曲線砍平了，讓使用量可以爆炸性增長而不擔心用戶回頭看帳單。從這個月開始，Claude Code 的 DAU 開始出現 Anthropic 後來描述為「垂直」的增長曲線。

這個決策的更深影響在一年之後才會顯現——Max Plan 成為 Claude Code 和 Cursor 在用戶經濟學上的分水嶺。Cursor 當時的 $20 Pro 以 request count 計費，Ultra $200 是很晚才加的。Max Plan 等於 Anthropic 提前一年在訂閱層級上站穩了「重度用戶友善」的位置。

2.5 Code with Claude：GA 大爆發（2025-05-22）

2025 年 5 月 22 日是 Claude Code 歷史上最密集的一天。

Anthropic 在舊金山 The Midway 舉辦了第一屆開發者大會「Code with Claude」。當天發布清單：

1. Claude Code 從 research preview 轉 **GA**
2. Claude **Opus 4**（$15/$75）+ Claude **Sonnet 4**（$3/$15）發布，成為 Claude Code 新預設模型
3. Claude Code 的 **VS Code 擴充**（beta）首發，帶 inline diff
4. **JetBrains 擴充**（beta）同時登場
5. **GitHub Actions** 整合（beta），透過 `/install-github-app` 安裝
6. **Claude Code SDK** 發布（幾個月後改名 Agent SDK）
7. **Parallel tool execution** 上線（LLM 可以一次 call 多個 tool 並行執行）

現場最讓人印象深刻的 demo，是 Claude Code 連續自主執行**90 分鐘**修復一個 Excalidraw 2022 年的 GitHub issue。這不是預錄的影片，是現場操作。90 分鐘裡它 grep、讀碼、改檔、跑測試、再改再跑，最後 commit 出一個能通過 review 的 PR。

Simon Willison 在現場直播部落格裡寫：「Claude Code is no longer a CLI — it's an ecosystem.」他的直覺是對的。這天之後 Claude Code 快速從一個「有趣的 CLI」變成一個「有多個客戶端、有 SDK、有 CI 整合、有訂閱計畫」的完整產品線。

GA 的第一版大致是 v1.0.0（雖然 Anthropic 後來的 release notes 裡沒有明確標這個版本號的發布日）。6 月 11 日我們可以在 npm 上看到 v1.0.23。從那時開始到 9 月底，Claude Code 停留在 v1.x 系列迭代了整整四個月，累積約 82 個 patch。這段時間的主軸是穩定性——新功能加得少，bug 修得多。

但中間有一個例外：hooks。

2.6 Hooks：從 prompt 到 lifecycle 的哲學轉向（2025-06）

2025 年 6 月（確切日期未見於 Anthropic 的一手 release note，但多個二手來源都指向這個月），Claude Code 推出 hooks 系統。

最早的 hooks 只有三個事件：PreToolUse、PostToolUse、Stop。功能很簡單——你在 `.claude/settings.json` 裡註冊一個 shell 命令，事件發生時系統自動跑它。表面上看這是個「小功能」，但它揭示了 Anthropic 對 agent 工程的一個深刻判斷：

最好的 agent 指令不是寫在 prompt 裡，是寫在 lifecycle 裡。

過去幾個月，社群解決「agent 每次都忘了 run linter」的做法，是在 CLAUDE.md 或 system prompt 裡寫一句「記得改完 code 要跑 eslint」。這個做法有兩個問題：一是 prompt 太長會被壓縮，二是「記得」這件事對 LLM 本來就不可靠。

Hooks 換了一個角度解決：linter 不是 agent 記不記得的事，是系統幫 agent 自動執行的事。PreToolUse 可以在 Edit 之前跑權限檢查，PostToolUse 可以在 Edit 之後自動跑 Prettier，Stop 可以在 session 結束時自動跑完整 test suite。

這是把「引導 agent」的職責從 prompt engineering 搬到了 system engineering。Anthropic 自己內部也用 hooks 跑 `console.log` 檢查、用它在 session 結束時 commit daily notes——後來公開的使用教學直接承認這點。

Hooks 在 6 月首發之後持續擴張。到 2026 年 Q1，hook 事件從 3 個增加到 17 個，涵蓋 PreCompact（壓縮前）、SessionStart、UserPromptSubmit、Notification 等所有 agent lifecycle 的關鍵節點。

2.7 Sub-agents：從單兵到 Task Runtime（2025-07-25）

2025 年 7 月 25 日，Claude Code 推出 custom sub-agents。

Sub-agents 的文件定義是「有獨立 context window、system prompt、tool 白名單的專用 agent」。你在 `.claude/agents/` 目錄放一個 markdown 檔案，frontmatter 定義 metadata，body 是 system prompt。主 agent 可以透過 Task tool 或 `/agents` 指令呼叫它。

這個機制看起來像是解決「context 爆炸」的一個工程優化——把耗 context 的子任務（比如讀 50 個檔案做 audit）隔離到 sub-agent 裡跑，主 agent 只看到最終摘要。但它的實際意義遠超這個解釋。

Sid Bidasaria 是 sub-agents 的主要設計者（他在 Pragmatic Engineer 訪談裡確認）。他設計 sub-agents 的核心判斷是：**agent 的未來不是一個越來越強的 agent，而是一群可以互相協作的 agent。**

這個判斷在兩個月後的 v2.0 發布時被驗證。v2.0 引入了七種 task 類型——LocalAgent、RemoteAgent、Dream、InProcessTeammate、LocalShell、MonitorMcp、Workflow——全部共享統一的生命週期狀態機（pending → running → completed/failed/killed）。這意味著 Claude Code 從此不再是一個 agent，而是一個「task runtime」，各種不同用途的 agent 都以 task 的形式存在。

Sub-agents 還做了一件事：它讓 Claude Code 變得 **cost-aware**。一個 sub-agent 的 frontmatter 可以指定用便宜的 Haiku 跑簡單任務，只在需要的時候才升級到 Opus。這是後來 Smart Model Switching 和 Auto Mode 的前身——Claude Code 從 v2.0 開始的所有模型切換邏輯都能追溯到 sub-agents 這一步。

2.8 v2.0 大改版：Sonnet 4.5 + Agent SDK + Checkpointing（2025-09-29）

2025 年 9 月 29 日是 Claude Code 的另一個里程碑日。

這一天 Anthropic 同時發布了三件大事：**Claude Code v2.0**、**Claude Sonnet 4.5**、**Claude Agent SDK v0.1.0**（從 Claude Code SDK 改名）。

v2.0 的三個旗艦特性：

Checkpointing——這是 v2.0 最被社群討論的功能。每次 user prompt 前系統自動做一個快照，你可以按 Esc Esc 或用 `/rewind` 回到任一歷史狀態，有三種模式（只回滾 chat、只回滾 code、兩者都回滾）。它解決了一個非常真實的痛點：agent 跑 30 分鐘後搞壞了一堆檔案，過去你只能 git reset，現在你可以「倒帶」到 30 分鐘前，換個 prompt 再跑。

原生 VS Code extension（GA 版）——5 月的 beta 只是把 Claude Code 嵌進 IDE，v2.0 的正式版加了 sidebar、即時 inline diff、collapsed tool result。從此 Claude Code 有了「IDE 形態」這個第二形象。

Terminal UI 重構——Ctrl+R 搜歷史 prompt、更好的 diff 顯示、改進的 streaming。這些是小改動，但每天用幾十次的使用者會立刻感覺到。

Sonnet 4.5 是這天的另一個重頭戲。Anthropic 宣稱它可以**連續自主工作 30 小時**（Opus 4 先前是 7 小時），OSWorld benchmark 從 42.2% 跳到 61.4%。這個「30 小時」後來成為 Claude Code 2025 Q4 的標誌性數字——它讓「睡覺前丟一個任務，早上起來看結果」第一次變成真實的工作模式。

Agent SDK 的改名是整天最容易被低估的決策。SDK 原本叫「Claude Code SDK」，改成「Claude Agent SDK」，從命名上就把邊界打開了——它不再只給寫 coding agent 的人用，任何想做 agent 的人都該用它。Anthropic engineering blog 在介紹文裡直接寫：「the SDK provides the same core tools, context management, and permission framework that Claude Code uses」——意思是「Claude Code 是 Agent SDK 最成熟的 reference implementation，但它不是唯一用途」。

這個改名的戰略意義巨大。它把 Anthropic 的商業故事從「賣 API token」擴展到「賣 agent platform」。之後幾個月，社群出現大量非 coding 的 Agent SDK 使用者：legal assistant、SRE bot、trading agent、security reviewer。Agent SDK 開始享受網路效應——因為 Claude Code 本身是它最強的 reference，任何學 Agent SDK 的人都能從 Claude Code 的源碼和行為推理出怎麼用 SDK。

2.9 Plugins / Skills / Web：生態大爆炸（2025 Q4）

2025 年 10 月是 Claude Code 生態的爆炸月，三個發布層層疊加。

10 月 9 日：Plugins（public beta）發布。 你可以把 slash commands、sub-agents、MCP servers、hooks、skills 打包進一個 plugin，透過 `/plugin` 指令安裝。Marketplace 是一個 git repo 加上 `.claude-plugin/marketplace.json` manifest——任何人都能 host 自己的 marketplace，Anthropic 官方 marketplace 叫 `anthropics/claude-plugins-official`，預設掛載，自帶 33 個官方 plugins 加 68 個合作夥伴 plugins（GitHub、Playwright、Supabase、Figma、Vercel、Linear、Sentry、Stripe、Firebase），到 2026 年 3 月達到 101 個。

10 月 16 日：Skills 系統正式發布 + Claude Haiku 4.5 發布。 Skills 的定位比 sub-agents 更輕巧——一個 skill 是一個 `SKILL.md` 加上資料夾，可以包含腳本、範例、參考資料。關鍵差別在於**觸發方式**：sub-agent 是被主 agent 明確呼叫（Task tool），skills 是 LLM 在對話中看到 description 匹配時自動載入（漸進式揭露，不占用 idle context）。Haiku 4.5 同日發布，極輕量、$0.8/$4 一 M token，Claude Code 在接下來幾週裡加入 smart model switching，開始把簡單任務自動分流給 Haiku。

10 月 20 日：Claude Code for Web（claude.ai/code）上線 + iOS app tab。 從此 Claude Code 不只是 CLI——你在瀏覽器打開 claude.ai/code，指向一個 GitHub repo，選擇 sandbox 環境（完全鎖、allow-list、或 `*`），丟 prompt，可以並行跑多個任務、切換 review。Mobile 上也能用。這是一個哲學上的轉變：**Claude Code 從「終端機裡的 agent」擴張為「多客戶端的 agent 平台」。**

這三個發布之間的內在邏輯很清楚。Plugins 解決「怎麼分享」——你有好的工作流，要能分享給別人。Skills 解決「怎麼自動選對工具」——系統要能在對話中自動匹配到適用的能力包。Web 解決「怎麼不受設備限制」——你坐在地鐵裡也能看 agent 的進度。合起來就是一個完整的生態平台。

11 月還有一個重要的月份里程碑：**Claude Code v2.1.0 發布**（Boris Cherny 在 Threads 親自發文公告，但沒有標具體日期）。累積 1,096 個 commits 的這個版本引入大量改進：Shift+Enter 換行零配置、hooks 可直接寫在 agent/skill frontmatter 裡、Skills 的 hot-reload + forked context + custom agent 支援、tool permission 的 wildcard 支援（`Bash(*-h*)`）、`/teleport` 把 session 搬到 claude.ai/code。這個版本把 Claude Code 從「功能齊全」推到「每個細節都打磨過」的階段。

2.10 Opus 4.5 與 66% 降價（2025-11-24）

2025 年 11 月 24 日發布的 Claude Opus 4.5 是個分水嶺事件。

這個模型本身是當時最強的 coding / agents / computer use 模型，SWE-bench Verified 在 80% 附近。但真正震撼的是價格——Opus 4.5 的價格定在 **$5/$25 per M tokens**，相比前一代 Opus 4.1 的 $15/$75，直接砍了 **66%**。同天發布的「Infinite Chats」讓 Claude Code 不再顯示 context limit 錯誤（背後是自動 compact）。

這個降價是 Anthropic 底氣的展現。能砍 66% 而不傷毛利，意味著 Claude Code / Agent SDK 的規模已經讓單位成本攤薄到能這樣做。從這個月開始，「Claude Code 重度用戶的月成本」這個經濟學命題被徹底改寫——一個每天跑 agent 跑 6-8 小時的工程師，月消費從 $800-$1,500 降到 $300-$500。

$5/$25 的價格還有一個副作用：它把 Anthropic 的 API 價格拉到和 OpenAI GPT-5 同一個量級，徹底堵死了「Claude 太貴所以用 GPT」這個口實。之後幾個月，Anthropic 多次公開財報數字暗示 Claude Code 的 ARR 翻倍速度。

12 月 18 日還有一個被低估的動作：**Agent Skills 被發布為 open standard（agentskills.io）**。Anthropic 把 Skills 規格標準化，允許 ChatGPT、Cursor 或其他平台採用同一份 spec 來執行同一個 skill。這在策略上把 Anthropic 自家的 agent instruction 格式推到 MCP 之後的第二個行業標準。

2.11 Opus 4.6 與 Agent Teams：1M Context 時代開啟（2026-02）

2026 年 2 月 5 日發布 Claude Opus 4.6，2 月 17 日接著是 Sonnet 4.6。這是 Anthropic 第一個真正意義上的 **1M context** 世代。

Opus 4.6 的三個關鍵功能：

1. **Agent Teams**——多個 agent 並行，各自擁有完整 1M context，透過「Mailbox Protocol」互傳訊息。這是 v2.0 sub-agents 和 Agent Swarms 功能的延伸，但規模和穩定性都上了一個台階
2. **Claude in PowerPoint**——Claude 可以直接操作 PowerPoint 檔案（computer use 的企業延伸）
3. **1M token context（beta）**——進入百萬 context 時代

Opus 4.6 被 Anthropic 在媒體溝通時打上「vibe working」的標籤——這是 Karpathy 2025 年發明的 vibe coding 一詞的企業版：讓 LLM 接手更完整的工作流程，人類只給方向和檢閱結果。

2026 年 2 月 19 日，Boris Cherny 上了 Lenny's Newsletter 的 podcast。這次訪談留下幾個後來被反覆引用的數字點：

- Claude Code 佔公開 GitHub commits 的 **4%**
- DAU 上個月翻倍
- Cherny 自己：「I have not written a single line of code since November」
- 他確認自己曾短暫離開去 Cursor 兩週後回歸 Anthropic

最後一點很有故事性——連 Claude Code 的創辦人都用過 Cursor，然後又回來了。這個細節在 X 上被反覆截圖討論，是 2026 年 Q1 兩派鬥爭最有象徵意義的一個註腳。

2026 年 3 月 13 日，1M context 正式 **GA**，而且關鍵是**沒有加價**——Opus 4.6 維持 $5/$25、Sonnet 4.6 維持 $3/$15 across the full window。其他 AI 公司（包括 Google 的 Gemini）都有 long-context premium，Anthropic 這個決策等於直接把 1M context 從「奢侈品」打成「基本配置」。Max / Team / Enterprise 訂閱者自動開啟，Pro 需要打 `/extra-usage` 主動 opt-in。

2.12 CronCreate 與 Background Workers（2026-03 初）

2026 年 3 月初，Claude Code 新增了三個 native tool：**CronCreate / CronDelete / CronList**，加上 `/loop` 和 `/schedule` 兩個 skill。

這是 Claude Code 第一次真正變成 **background worker**。過去 agent 只能在你開著 terminal 的時候跑——關了 terminal 任務就消失。CronCreate 讓你可以「每小時檢查一次 error log，如果有新 bug 自動開 PR 修復」，而且 agent 跑在 Anthropic 雲端，你的筆電關機也能跑。

Anthropic 工程師 Thariq Shihipar 在一篇 blog post 裡示範這個用法，社群的反應是兩極——一派覺得這是 Claude Code 終於進化成「真正的 agent」，另一派擔心這是「AI 自動化失控」的起點。兩邊都有道理，也都捕捉到 CronCreate 這個動作的重量級。

Schedule task 的實現有幾個刻意的限制：session-scoped（關閉 terminal 後如果是 session 任務會消失）；recurring task 三天後自動 expire（防止「寫完就忘了」的任務無限跑下去）；busy 時不 catch-up missed fires（不會積累）。這些限制都是在說「我們不希望你丟一個 agent 就永遠不管」——Anthropic 明顯在設計上對「完全無人值守」還保持謹慎。

2.13 地震月：源碼洩漏與安全連環（2026-03-31）

2026 年 3 月 31 日是 Claude Code 歷史上最戲劇性的一天。

事件的引爆是這樣：Anthropic 在發布 Claude Code v2.1.88 的 npm package 時，`.npmignore` 漏了一行——`.map` 檔案沒被排除。結果整個 source map **59.8 MB、513,000 行 TypeScript、1,906 個檔案**的源碼被意外公開到 npm 上。

研究員 Chaofan Shou 在幾小時內在 X 上公開這件事，並從 Anthropic 的 Cloudflare R2 bucket 直接下載 source map，mirror 到 GitHub。幾小時後一個叫 `Claw-code` 的 clean-slate rewrite 專案成立——這個專案在 48 小時內衝到 55,800 star，成為 **GitHub 歷史上最快達到 50k 星的 repo**。

洩漏出的源碼裡有很多令人尷尬的細節。比如有一個「undercover mode」system prompt，指示 Claude 在特定情境下不提自己是 AI、不讓 agent 在 commit 訊息裡署名外部 repo 的貢獻者。這被一些人批評為「有 dark pattern 嫌疑」。另外 v2.1.88 裡隱藏的 feature flag 系統（87 個 compile-time flag + 16 個 runtime GrowthBook flag）也第一次完整曝光——很多之前傳聞中的 Anthropic 內部功能（Agent Swarms、Computer Use、Voice Mode、Bridge/CCR）全部被 flag 門控，真的存在於源碼中。

洩漏只是開始。4 月連續發生兩件事：

1. **4 月 1 日**：Register、Trend Micro 等媒體報導，有人在 GitHub 上假冒 leaked-source 下載，內含 Vidar infostealer 和 GhostSocks 惡意軟體。大量開發者被釣
2. **4 月 2 日**：Check Point Research 披露 **CVE-2025-59536 / CVE-2026-21852**——透過 malicious project config（hooks / MCP servers / env vars）可以達到 RCE 和 API token exfiltration

如果這還不夠戲劇性，同一個月 SOCRadar 發布了一份震撼報告：2025 年 12 月到 2026 年 2 月的墨西哥政府大規模入侵事件中，攻擊者從聯邦、州、市三級偷走稅務、戶籍、健康、選舉、採購、基礎設施資料。分析顯示攻擊流程裡 **~75% 的 RCE 活動是 Claude Code 生成與執行的**。

這直接把 Claude Code 推上「是否該被視為犯罪武器」的爭議。Anthropic 的回應是快速推出一系列安全強化——permission 新增更嚴格的 enterprise 層、hooks 增加數位簽章選項、MCP server 的黑名單機制。但品牌傷害已經造成：Claude Code 作為「safety-first」的 Anthropic 最旗艦產品被成功武器化，這件事在 Anthropic 企業銷售敘事裡很難洗白。

2.14 Opus 4.7 反擊與 Desktop 重繪（2026-04）

2026 年 4 月是 Claude Code 的反彈月。

4 月 14 日：Claude Code Desktop App 大改版 + Routines（research preview）。Mac / Windows 桌面版完全重繪，新的 sidebar 管理多 session、drag-and-drop layout、整合終端機與檔案編輯器。同日推出 Routines——儲存的 Claude Code 自動化任務，跑在 Anthropic 雲端，筆電關機也能持續執行。

4 月 16 日：**Claude Opus 4.7 發布**，同步上 Claude Platform、Amazon Bedrock、Google Cloud Vertex AI、Microsoft Foundry。SWE-bench Verified 推到 **87.6%**（從 Opus 4.6 的 80.8% 提升），CursorBench 從 58% 提升到 70%，GPQA Diamond 94.2%。新功能包括 xhigh effort level、Task budgets、`/ultrareview` 多代理碼審、vision 升到 2,576 px / 3.75 MP。**1M context 價格依然維持 $5/$25**。

4 月 14-18 日一週內連續發布 v2.1.108 到 v2.1.114 的 patch 系列，引入一大串社群期待已久的功能：1-hour prompt cache TTL（`ENABLE_PROMPT_CACHING_1H`）、session recap、`/effort` interactive slider、Auto mode（在 Opus / Sonnet / Haiku 之間自動切，Max 訂戶專屬）、PowerShell 工具漸進 rollout、push notification tool、`EnterWorktree` 支援 path 參數、PreCompact hook、plugins 的 background monitor。

這一個月的反擊是精心安排的。用產品更新和模型能力把話題從 source leak 拉回來，讓市場看到「Anthropic 的技術護城河還在繼續拉大」。SWE-bench 87.6% 這個數字在 4 月下旬被到處引用，是 Claude Code 在 Codex CLI 的一波話題攻勢後拿回敘事主導權的關鍵一擊。

到 2026 年 Q1 結束，根據 Constellation Research 的數據，**Claude Code 的 ARR 從 2026 年 1 月 1 日到 4 月份翻倍**，對 Anthropic 整體營收的貢獻超過一半。Claude Code 已經不是 Anthropic 的一個產品線，它是 Anthropic 的主戰場。

2.15 階段總結：六個時期

回頭看這 18 個月的軌跡，Claude Code 的發展可以劃分為六個階段：

階段	時期	核心特徵	核心矛盾
萌芽期	2024 Q3-Q4	Boris 的週末 hack 成為內部爆款、MCP 協議奠基	「這是玩具還是產品」
Preview 期	2025 Q1-Q2	research preview 公開、快速 v0.2.x 迭代	「能不能 GA、要不要收費」
起飛期	2025 Q2-Q3	GA、Max Plan、VS Code/JetBrains、Opus 4/Sonnet 4	「穩定性 vs 新功能」

階段	時期	核心特徵	核心矛盾
平台化	2025 Q3-Q4	v2.0、Sub-agents、SDK 改名、Plugins、Skills、Web	「做產品還是做平台」
規模化	2025 Q4 - 2026 Q1	Opus 4.5 降價 66%、Opus 4.6 1M context、CronCreate	「scale 與成本的張力」
地震期	2026 Q1 末	Source leak、安全事件、品質爭議、Opus 4.7 反擊	「信任 vs 領先」

每個階段的時長在壓縮——從第一階段的四個月，到第五階段的兩個月。這說明 Claude Code 已經進入「季度級別的戰略節奏」：每兩三個月必須推出一個能改變敘事的新功能，否則對手就會趕上。2026 年 Q2 的節奏只會繼續加速。

三、原理剖析：Claude Code v2.1.88 的內部架構

這一節基於 2026 年 3 月 31 日意外洩漏的 Claude Code v2.1.88 源碼還原。原始碼規模：1,884 個 TypeScript/TSX 檔案、37 個頂層目錄、40+ built-in tools、88+ slash commands、87 個 compile-time feature flags、16 個 runtime flag、7 種 task type。

3.1 七層架構：整體骨架

Claude Code 的架構可以分成七層，從外到內：

層	名稱	職責	代表檔案
1	**Entry Points**	CLI / MCP / SDK 三個進入口	`entrypoints/cli.tsx`、`entrypoints/mcp.ts`、`entrypoints/init.ts`
2	**Screens**	主互動介面（REPL 為主）	`REPL.tsx`（895KB）、`Doctor.tsx`、`ResumeConversation.tsx`
3	**Core Pipeline**	query loop + tool registry + command dispatch	`query.ts`、`tools.ts`、`commands.ts`、`context.ts`
4	**Agents**	七種 task 類型的 runtime	`agents/local-agent.ts`、`agents/teammate-runner.ts`、`swarms/*`

層	名稱	職責	代表檔案
5	**Tools**	40+ built-in tools	`tools/Read/`、`tools/Bash/`、`tools/Edit/`、`tools/Task/`
6	**MCP**	7+ transport 的 MCP client	`mcp/transport/*`、`mcp/connection-manager.tsx`
7	**Extensibility**	Commands + Skills + Plugins 三軸擴展	`commands/*`、`skills/*`、`plugins/*`

七層中最令人意外的是第二層——Claude Code 的 UI 是 React/Ink 寫的，REPL.tsx 單檔 895KB，裡面 389 個 React 組件、104 個 hooks。這個規模在 CLI 工具史上前所未見，它把一個終端機當作一個「受限的瀏覽器」來開發。

3.2 啟動效能：Fire-and-Await 的巧妙設計

CLI 的第一印象是啟動速度。Node.js 的冷啟動本身就慢，加上 Claude Code 要載入 1,884 個檔案的 dependency graph，天真做法會讓啟動時間來到 3-5 秒——對 CLI 工具是不能接受的。

Claude Code 的解法叫 **Fire-and-Await**，核心在 `main.tsx`：

> 在 module evaluation 時就 fire 一批 async I/O（讀 MDM 設定、預抓 keychain），到 `setup()` 函式被呼叫時再 await。中間的 module graph 載入時間「藏」進了 I/O 等待裡。

這不是 top-level await。Top-level await 會阻塞所有 import 此模組的下游檔案，反而讓並行度變差。Fire-and-Await 用純 Promise——I/O 在背景跑，不阻塞任何東西，要結果的時候才 await。

`setup()` 內部再用一次並行策略：三個互相獨立的初始化（`getExampleCommands`、`getContext`、`initializeStatsig`）用 `Promise.all` 同時跑，只有在「B 依賴 A 的結果」時才序列化（`resolveFeatureFlags` 依賴 `initializeStatsig`、`loadModelCapabilities` 依賴 featureFlags）。

這種「手工設計依賴圖、最大化並行」的工程紀律，把一個理論上 3-5 秒的啟動壓縮到 400-800ms——使用者幾乎沒感覺。

3.3 Query Pipeline：AsyncGenerator 作為核心抽象

Claude Code 的心臟是 `query()` 函式，它是一個 `async function*`（AsyncGenerator）。使用者的每個 prompt 都進入這個 generator，被轉化成一連串事件流。

為什麼選 AsyncGenerator 而不是 RxJS Observable 或 EventEmitter？因為它完美契合四個需求：

需求	AsyncGenerator 的解法
串流	每個 `yield` 推送一個事件給消費端
工具迴圈	Generator 內部可以遞迴呼叫自己
UI 同步	消費端用 `for await` 接收，天然是非同步的
可中斷	`generator.return()` 或 AbortController 一行搞定

RxJS 可以做到同樣的事，但要 50KB 額外依賴、operator 鏈的 TypeScript narrowing 不夠乾淨。EventEmitter 沒有 backpressure——buffer 會爆炸。AsyncGenerator 零依賴、型別友善、語義清晰——**最簡單能解決問題的方案就是最好的方案**。

query loop 的實際邏輯大致是這樣：

```
for each turn:
  1. 組裝 messages (system prompt + history + latest user input)
  2. 呼叫 Anthropic API (streaming)
  3. 解析回傳中的 tool_use blocks
  4. 若無 tool_use → yield final answer、結束
  5. 若有 tool_use → 並行執行所有 tool、把 tool results 追加到 messages
  6. 回到步驟 1
```

整個迴圈平均跑 8-15 次（一個簡單任務 2-3 次、一個複雜 refactor 40+ 次），每一輪都能透過 AsyncGenerator yield 即時事件給 UI 層。這個模式是 Claude Code 體感流暢的根本原因——使用者看到的不是「等 2 分鐘然後一口氣出結果」，而是 agent 每一步都有回饋。

3.4 Tool System：satisfies 的隱式 Plugin 架構

Claude Code 有 40+ built-in tools，加上 MCP 動態載入的工具可以無限擴展。但令人意外的是，它**沒有正式的 Plugin SDK**——沒有 `AbstractTool` 基類、沒有 `registerTool()` 全域函式、沒有 decorator。

每個 tool 是一個物件字面量，用 TypeScript 4.9 新加的 `satisfies` 關鍵字宣告：

```
export const ReadTool = {
  name: "Read",
  description: "...",
```

```
  inputSchema: { ... },
  call: async (input, context) => { ... },
  checkPermissions: async (input, context) => { ... }
} satisfies ToolDef
```

`satisfies` 的妙處在於它**檢查型別而不拓寬**——`name` 欄位被推導為字面量型別 `"Read"` 而不是 `string`，這讓下游的 discriminated union 能正確運作。同時 tree-shaking 友善、單元測試無痛、零 boilerplate。

Anthropic 沒有做正式的 plugin 系統，但這個「物件字面量 + satisfies」的慣例本身就是一個 plugin 系統——**隱式**的 plugin 架構。每個 tool 都有統一的結構（name、description、inputSchema、call、checkPermissions），但不需要繼承基類；新增 tool 只要加一個 export，bundler 自動處理 registration。

這是一種刻意的反框架哲學。Anthropic 選擇了 **慣例**而不是**框架**——用 TypeScript 型別系統做 contract、用 bundler 做 registration、用 code convention 做 extension point。代價是沒有 runtime introspection（不能動態發現 tool 有哪些欄位可用），但這個成本對一個內部控制的 codebase 無關緊要。

3.5 Permission Model：八層 Cascade 的縱深防禦

Claude Code 是能讀寫檔案、執行 shell、存取網路的 agent。權限模型的失守代價很高——`rm -rf /` 不是開玩笑的。

Anthropic 的做法是 **八層 cascade**：

層	來源	優先級
1	CLI arguments	最高
2	Session-level decisions（你在 UI 點「Allow」）	↑
3	Command/Skill 內建規則	↑
4	Enterprise policy（IT 管理員）	↑
5	Local settings（`.claude/settings.local.json`）	↑
6	Project settings（checked into repo）	↑
7	User settings（`~/.claude/settings.json`）	↑

層	來源	優先級
8	Default（**Fail-Closed**）	最低

高層覆蓋低層。任何一個規則不只是「allow / deny」，而是一個帶 **source tracking** 的結構——當使用者問「為什麼不能跑這個命令」，系統能精確回答「因為 project settings 裡有一條 deny 規則」，而不是一堆猜測。

第 8 層的 **Fail-Closed** 是核心哲學：任何模糊地帶預設拒絕。要繞過整個系統，你必須在 CLI flag 裡打 `--dangerously-skip-permissions`——這個名字本身就是社會工程防禦，不可能「不小心」輸入。

八層的真正意義是信任鏈的顯式化：Layer 1-2 是當前使用者的即時意圖、Layer 3-4 是組織/平台政策、Layer 5-7 是歷史偏好、Layer 8 是安全底線。每一層對應一個不同的「信任來源」。

3.6 Agent Architecture：七種 Task、統一生命週期

v2.0 之後 Claude Code 的 agent 系統不再是「單 agent + sub-agent」，而是七種 task 類型：

Task 類型	用途	隔離度
LocalAgentTask	本地 sub-agent、同 process fork	共享 process
RemoteAgentTask	CCR 遠端任務（ultraplan、ultrareview、autofix-pr 等）	完全獨立
DreamTask	背景記憶整理	同 process 但獨立 context
InProcessTeammateTask	Agent Swarms 的同 process 隊友	AsyncLocalStorage 隔離
LocalShellTask	長時間 shell 命令（不涉 LLM）	OS process
MonitorMcpTask	MCP server 的健康監控	獨立監控通道
WorkflowTask	跨 task 的 orchestration	編排器角色

七種完全不同的 task，共享一個統一的生命週期狀態機：`pending → running → completed | failed | killed`。這讓 UI 能用一個 TaskList 顯示全部、讓 cleanup 邏輯能一次處理所有異常、讓計費/監控邏輯能統一追蹤。

120 秒自動背景化是 LocalAgentTask 的精妙設計——大多數 sub-agent 的工作 30-60 秒完成，超過 120 秒的通常是複雜任務，使用者不該乾等。自動背景化後主 agent 可以繼續互動，或同時啟動多個 sub-agent 並行。

Agent Swarms 是 2025 年 Q4 引入、2026 Q1 穩定的功能。它的核心是一個 leader + 多個 teammates + 一個 mailbox 的結構。Teammates 可以跑在三種 backend 上：

- **InProcessBackend**：AsyncLocalStorage 隔離，零啟動成本，共享 MCP server
- **TmuxBackend**：新 tmux pane 中獨立 process，完全隔離、使用者可切過去看
- **ITermBackend**：macOS iTerm2 native split pane（AppleScript API）

三種 backend 功能相同，隔離度和啟動成本遞增。系統自動偵測：在 tmux session 裡優先 Tmux、macOS 上有 iTerm2 優先 ITerm、fallback 到 InProcess。

3.7 Context 管理：六層壓縮 + 五層記憶

LLM 應用最貴的稀缺資源是 context window。Claude Code 用兩套機制管理：六層壓縮 + 五層記憶。

六層壓縮（從細到粗）：

層	策略	觸發
1	**Micro Compaction**	單個 tool output 超過 `maxResultSizeChars`，保頭保尾截中間
2	**Auto Compaction**	剩餘 token < 13K，LLM 做結構化摘要
3	**Reactive Compaction**	API 回 `context_length_exceeded`，激進壓縮只保最近 3 輪
4	**Snip**	標記單條訊息為可遺忘，內容替換為 `[snip]` 但保留位置
5	**Context Collapse**	分 anchor / middle / recent 三段，middle 全部 LLM 摘要成一條
6	**Manual Compaction**	使用者 `/compact [focus]` 主動觸發，可指定保留焦點

每一層都在「資訊保留」和「空間回收」之間做不同取捨。Micro 最便宜但粒度最細，Collapse 最貴但能釋放最多空間。Anthropic 內部做過大量調參，13K auto compact threshold 不是亂選的——它等於「一次 LLM 回應（~4K）+ 一次 tool result（~8K）+ 安全邊際（~1K）」，剛好夠 agent 跑完當前 turn。

五層記憶：

層	名稱	持久性	範圍
1	**CLAUDE.md**	磁碟、git 追蹤	專案級（巢狀載入）
2	**Auto Memory**	`~/.claude/memory/`	使用者級、跨 session
3	**Session Memory**	in-memory	單一 session
4	**Extract Memories**	即時提取的精華	Session 內
5	**Team Memory Sync**	共享儲存	團隊同步（`local_wins` 策略）

這五層覆蓋不同的時間尺度。CLAUDE.md 是「專案憲法」，改動需要 commit；Auto Memory 是「個人筆記」，使用者或系統寫入；Session Memory 是「工作記憶」，跨 compaction 存活但不跨 session；Extract Memories 是對話中即時提取的「學習」；Team Memory 是多人協作的同步層。

Context 組裝時的順序很講究：**使用者指令放最前（primacy effect）、系統 context 放最後、中間去重**。`filterDuplicateMemoryAttachments` 用一個 Set 做 O(1) 去重——同一條規則可能同時出現在 CLAUDE.md、rules 檔案、Auto Memory 裡，不去重會佔三倍空間。

3.8 擴展性三軸：Commands / Skills / Plugins

Claude Code 的擴展系統有三個獨立的軸，服務不同的消費者：

第一軸：Commands（指令系統）——88+ slash commands，服務「人類使用者的快速動作觸發」。每個 command 實作 `name`、`aliases`、`description`、`execute(args, context)`。特別的是 `execute` 可以回傳文字**或 JSX 元件**——因為 UI 是 React/Ink，你可以直接 return 一個帶色彩的表格組件。這是 CLI 工具中罕見的設計。

第二軸：Skills（技能系統）——服務「LLM agent 的語意化能力描述」。每個 skill 是一個 markdown 檔案加 frontmatter，主 agent 透過 LLM 語意判斷是否適用。關鍵差異是 Skills **不占用 idle context**——只有在對話匹配到相關語境時才載入。

發現邏輯：先 tokenize 使用者訊息做 keyword 比對、失敗就用 regex pattern 匹配、多個命中就依匹配強度排序、把「Found applicable skills: ...」注入 system prompt 讓 LLM 決定是否使用。為什麼不直接自動啟用？因為每個 skill 的 context 消耗 3-5K tokens——false positive 代價太大。**只建議不強制，讓 LLM 做最終判斷**。

第三軸：Plugins（外掛系統）——最重量級，基於 MCP 協議。一個 plugin 是一個 git repo，可以打包 commands、sub-agents、MCP servers、hooks、skills。透過 `/plugin` 安裝，marketplace 可以是任何 git 倉庫 + 一個 manifest JSON。

三軸之間的關係不是並列，是**補充**。Commands 是最快的觸發；Skills 是最省 context 的能力注入；Plugins 是最完整的分發。使用者從 Commands（零學習曲線）到 Skills（開始寫 markdown）到 Plugins（封裝完整工作流），認知負擔逐步增加，但能力也逐步擴展。

3.9 MCP 整合：從協議規格到生產系統

MCP 這個協議的 spec 其實很簡單——它只定義訊息格式（tools、resources、prompts）和兩種基本 transport（stdio 和 HTTP+SSE）。但 Claude Code 把它擴展到 **7+ 種 transport**：

STDIO、SSE、SSE_IDE（IDE 特化）、HTTP（new streamable）、WS、IN_PROCESS（同進程函式呼叫）、SDK_CONTROL（Anthropic 內部）。

IN_PROCESS 是很妙的優化——某些內建服務（如 LSP）不需要跨進程通訊，直接同 Node.js process 內函式呼叫，延遲從毫秒級降到微秒級。使用者不需要知道這件事，系統自動判斷：有 `command` → STDIO、有 `url` → 看 protocol、`inProcess: true` → IN_PROCESS。

MCP 的配置還有 **7 層 scope**：local / user / project / dynamic / enterprise / claudeai / managed。低層覆蓋高層，但 enterprise scope 可以設 `blocked` 強制禁用—— `blocked` 是 fail-closed 的延伸，一旦設定就是不可覆蓋的黑名單。

連線管理包在 `MCPConnectionManager` 裡，這是一個 React Context——MCP 狀態自動觸發 UI re-render，斷線時相關 UI 組件自動顯示「不可用」。這是架構設計的一個巧妙體現：MCP 狀態天然屬於 React 生命週期（UI 驅動），所以不用 bridge 到非 React 世界。

3.10 React/Ink UI 與 Closure Bridge

Claude Code 選擇 React/Ink 做終端 UI，這個決定的代價是：React 的狀態管理和 agent runtime 的狀態是兩個不同世界。

React/Ink 裡狀態靠 hooks、re-render、immutable state。Agent runtime 裡狀態是純 TypeScript 的 async 函式，沒有 React context、沒有 hooks。兩者必須共享狀態——agent 執行進度要顯示在 UI、UI 的權限決策要傳回 agent。

解法是 **Closure Bridge 模式**：

```
  React Tree                    Agent Runtime
 +-------------+              +-------------+
 | useState()  |              |  async      |
 |  ref.set()  | <-----+      |  function   |
 |  ref.get()  |       |      |             |
 +-------------+       |      +-------------+
       ^               |            |
       |               |            v
```

```
useSyncExternalStore  |    registerGlobalBridge(ref)
        ^             |          |
        |             |          v
+-------------+       |   +-------------+
| store.ts    | <-----+---|  closure    |
|  34 lines   |           |  holds ref  |
+-------------+           +-------------+
```

REPL.tsx 在 mount 時 `registerGlobalBridge()` 把 React 的 state ref 交給 agent runtime 的 closure。Agent 要改 state 時透過 ref.set()，React 用 `useSyncExternalStore` 訂閱變化自動 re-render。這是一個「雙向門」——兩個世界各自乾淨，只通過明確的 bridge 溝通。

狀態切兩個 store：AppState（對話級、高頻）和 BootstrapState（session 級、低頻）。這是為了避免 `totalCostUSD` 每次增加都觸發所有訂閱 `messages` 的組件 re-render。

最驚人的細節是：Claude Code 沒用 Zustand 或 Jotai，而是自己寫了一個 **34 行的 store**。`getState` 回傳 state 物件本身（非 deep copy），消費者可以直接 mutate——Claude Code 靠程式碼紀律防止 mutation，所有更新都透過 spread operator。這在團隊規模小的場景下可行，但若用於開源大型團隊，建議加 `Object.freeze()`。

四、橫向分析：2026 Q2 的競爭圖譜

4.1 市場地圖

2026 年 Q2 的 AI 編碼工具市場，已經分化出三種清晰的產品形態：

1. **IDE 路線**：Cursor、Windsurf、Zed、JetBrains Junie——把 AI 塞進編輯器，打磨 inline 體驗
2. **終端路線**：Claude Code、Codex CLI、Aider、Gemini CLI、Amp、GitHub Copilot CLI——把編碼當 agent loop 跑在 shell 裡
3. **雲端自主路線**：Devin、Replit Agent、GitHub Copilot Coding Agent——你發 ticket，它還 PR

Claude Code 屬於第二類，但它最大的對手（Cursor）屬於第一類。這是 2026 年最有趣的一組對抗：**終端派 vs IDE 派，誰是未來編碼工具的主入口?**

營收層面的快照：

- Anthropic 總 ARR 約 $30B 量級（Series G 估值 $380B），Claude Code 貢獻 > $2.5B，佔企業支出一半以上。DAU 自 2026 年 1 月 1 日翻倍
- Cursor（Anysphere）ARR 約 $2B、估值 $29.3B（2025 年 11 月輪），2026 年 4 月傳出在談 $50B 的新輪

- Cognition（現在擁有 Windsurf + Devin）、OpenAI（Codex CLI 主力）緊追
- Cline 累積 500 萬安裝、5.8 萬 star，拿到 $32M Series A+Seed

4.2 Cursor：最大的對手

Cursor 是 2026 年 Claude Code 的核心對手，也是它在哲學上的完美反面。

Anysphere 這家公司從 2024 年的一個 MIT 四人小組，到 2026 年成為 AI coding 賽道最值錢的私人公司。成長曲線在 SaaS 史上也是罕見的——2025 年 1 月 ARR $100M、6 月 $500M、11 月 $1B、2026 年 2 月 $2B。**被稱為「歷史上成長最快的 B2B SaaS」**，超越了 Slack、Snowflake、Zoom 的同期速度。2025 年 11 月拿到 $2.3B、估值 $29.3B（Accel、Coatue 領投）。2026 年 4 月傳出正在募 $2B+、估值 $50B、A16z 與 Thrive Capital 領投、Nvidia 戰略跟投。

Cursor 的設計哲學可以一句話概括：**開發者仍然坐在編輯器前，AI 是加速器而不是代辦人**。這個哲學的兩個招牌功能：

- **Tab autocomplete**——sub-second 的 next-edit prediction。它不只補當前行，還預測你下一個要編輯的位置並引導光標跳過去。2026 年被普遍認為是業界最強的 inline 補全
- **Composer**——多檔案 agent mode。Composer 2 是 Cursor 自家訓練的模型（不是單純 call Claude/GPT），針對 IDE 工作流優化

Cursor 的模型策略是**多模型支援但有自家模型**——用戶可以在 Claude、GPT、Gemini 之間切換，但 Cursor 推自己的 Composer 以降低對 Anthropic/OpenAI 的依賴。這是典型的「不綁死」策略，長期看降低了供應商風險。

Cursor 的使用者口碑兩極化。支持派最常提的三個理由：Tab 補全是「最接近讀心術」的體驗、上手最快（打開就能用）、模型切換自由。槽點也很集中：資源重（VS Code fork 吃記憶體）、代碼品質盲測落後 Claude Code（多個第三方測試顯示 rework 率高 30%）、在長任務中「第一個模糊決策點就停下來」。

Cursor 贏 Claude Code 的地方：Day 1 生產力、Tab 補全（Claude Code 終端裡沒等價物）、視覺化編輯、模型多樣性、UI 生成（Cursor 3 的 Design Mode）。

Cursor 輸 Claude Code 的地方：代碼品質（盲測勝率低）、token 效率（Claude Code 用 5.5x 更少 token）、長任務自主能力、資源佔用。

社群在 2026 年 Q1 形成的共識是：**最強生產力組合是兩個一起用**——Claude Code 負責 heavy lifting（refactor、feature、test），Cursor 負責 daily grind（inline editing、UI tweak）。這個組合被多篇 2026 年文章稱為「the 2026 stack」。

一個標誌性的細節是 Boris Cherny 自己在 Lenny's Newsletter 承認他 2025 年末曾短暫離開去 Cursor 兩週——然後回 Anthropic。這件事被 X 反覆引用：**連 Claude Code 的創辦人都用過 Cursor**，這個事實讓兩派之爭更有趣，也更實在。

4.3 Cline：開源功能鏡像

Cline（原名 Claude Dev）是 Claude Code 最直接的功能鏡像——但開源、BYOK、活在 VS Code 裡。

2024 年中 Saoud Rizwan 以 Claude Dev 為名發布這個 VS Code extension，2024 下半年改名 Cline。完全開源（Apache 2.0）、BYOK（你付 Anthropic / OpenAI / OpenRouter 的 API 費，不付 Cline 訂閱費）。工具本身免費。2026 年中拿到 **$32M Series A+Seed**，angels 包含 Jared Friedman、Logan Kilpatrick（前 Google AI DevRel 負責人）、Addy Osmani。累積 500 萬安裝、5.8 萬 stars、6000+ forks。

Cline 的核心設計哲學：**Claude Code 能做的，我在 VS Code 裡也能做，而且開源、BYOK、完全透明**。

最大的哲學差異是 **Plan/Act 雙模式**——每個操作都要人類批准（file edit、terminal command 都先 preview）。這個 human-in-the-loop 是 Cline 與 Claude Code 最大的差異：Claude Code 在 2026 年預設已經 auto-approve 很多操作，Cline 堅持每一步都要你點確認。

Cline 和 Claude Code 最像的地方是 MCP 支援——同樣用 MCP 協議擴展外部工具。2026 年 Cline 還推出了 **Cline Kanban** 做多 agent 並行 orchestration，每個 task 在自己的 git worktree 跑。

Cline 的使用者口碑：支持派最愛完全透明（每個 API call、每個 token、每個改動都看得到）、VS Code 深度整合（diff view、Problems tab、Git integration）、BYOK 意味無限額焦慮。槽點也很集中——速度慢（社群共識「literally twice as slow as Cursor」）、被 Roo Code（Cline 的 fork）吃份額、746 個 open issues 暴露資源緊張、多檔案 refactor 精度只有 7/10。

Cline 贏 Claude Code 的地方：開源 + BYOK、完全透明、VS Code 深度整合、無限額焦慮。

Cline 輸 Claude Code 的地方：速度、自主性（預設 human-in-the-loop）、代碼品質、生態成熟度（Claude Code 的 skills/hooks/sub-agents 更完整）。

Cline 在 2026 年處在一個尷尬的位置：它的理念（開源、BYOK、透明）對 hacker 群體很吸引，但規模化用戶越來越多選 Claude Code 的免費 plugin marketplace 加上訂閱 bundle。Saoud 和團隊的生存策略是往團隊版 Cline Teams 和企業合規打。

4.4 Aider：元祖派的古典堅持

Aider 是 2023 年就有的 CLI 編碼工具，Paul Gauthier 一個人主導（2025 年專案改組成 `Aider-AI/aider`，開始社群化治理）。完全開源（Apache 2.0），純 CLI 純 Python，沒有 GUI、沒有 IDE 整合、沒有 Web UI。

Aider 是「Unix 哲學」在 AI 編碼時代的代言人：

1. **Git 是真相來源**——每個 AI 改動自動 commit，訊息由 AI 寫，回滾就是 `git reset`
2. **終端就是 UI**——不要 GUI、不要批准按鈕、不要左側面板

---

3. **把模型當工具**——支援幾乎所有主流模型（Claude、GPT、Gemini、DeepSeek、o1/o3、本地 Llama）
4. **編輯格式才是關鍵**——Aider 最獨特的設計是有多套 diff format（whole/diff/udiff/editor-diff/polyglot-diff）並根據模型特性切換

Architect Mode 是 Aider 最有影響力的設計：兩個模型分工——Architect 模型（如 o1、R1）負責思考解法，Editor 模型（如 Sonnet、DeepSeek-Coder）負責把解法轉成 diff。R1+Sonnet 組合曾在 Aider 自家 polyglot benchmark 達到 SOTA，成本只有 o1 solo 的 1/14。

Context 管理要使用者自己顯式 `/add` 檔案——沒有自動索引、沒有向量檢索。Paul 的哲學是「讓你知道模型看到什麼」。

Aider 的 polyglot benchmark 是業界事實標準之一。很多模型發布時都要貼 Aider score。Opus 4.7 和 GPT-5.4 的 coding benchmarks 裡，Aider polyglot 永遠是一個被引用的數字。

Aider 贏 Claude Code 的地方：token 效率（4.2x 更省，重度月 $60-80 vs Claude Code $200+）、Git integration 深度（每個動作自動 commit）、模型靈活性（本地 Llama、Qwen、DeepSeek 一鍵切）、benchmark 透明性。

Aider 輸 Claude Code 的地方：自主 agent 能力（沒有 sub-agent、hook、skill 的類比）、MCP 生態、Day 1 體驗（新手基本必須看文件）、自動發現檔案（Aider 要你手動 `/add`）。

歷史地位總結：Aider 是 CLI AI 編碼的先驅，留給 2026 年的遺產是兩件事——polyglot benchmark 成為行業事實標準、「多 diff format + Architect/Editor 分工」的設計思想被後來者廣泛採納（Claude Code 的 plan mode、Codex CLI 的 planning agent 都有它的影子）。它活成了「冷門但在技術圈受尊敬」的那種工具——像 vim 之於 VS Code。

4.5 Codex CLI：正面對撞的 OpenAI 旗艦

警告：此 Codex ≠ 2021 年支援 GitHub Copilot 的舊 Codex 模型。這是 2025 年 4 月 OpenAI 重啟的新產品線，共享名字、不共享技術棧。

Codex CLI 是 OpenAI 直接對標 Claude Code 的產品。純 CLI，用 **Rust** 實作（這是它和 Node.js 的 Claude Code 的根本差異）。完全開源（Apache 2.0），這和 Claude Code 的閉源形成明顯對比。

版本節奏驚人——12 個月 **696 個 releases**，接近每天兩個版本（shareuhack.com 2026 年 4 月）。這個迭代速度在開源 CLI 專案裡前所未見。75K+ GitHub stars、月均 npm 下載 1,450 萬、**週活用戶 300 萬**（Sam Altman 2026 年 4 月 8 日親自確認）。

2025 年 6 月發生的一個關鍵決策：**從 TypeScript 全面重寫為 Rust**。今天的 codebase 95.6% Rust。這帶來兩個明顯優勢：啟動時間 < 100ms（Claude Code 通常 1-2 秒）、Linux 上用 Landlock/seccomp 做原生沙箱（Node.js 版本做不到）。

Codex CLI 的設計哲學：**Claude Code 做的事，我們要做得更快、更省 token、更自主**。

- **GPT-5-Codex**：為 agentic coding 專門訓練的 GPT-5 變體，「長任務中保持連貫性」是官方主打
- **Full-auto mode 哲學**——預設就可以無審批執行（和 Cline 的 human-in-the-loop 完全相反）。OpenAI 相信「你應該 fire-and-forget，回來看結果」
- **Multi-variant generation**（2026 新功能）——一個 task 生成 2-4 個不同實作，讓你挑哪個繼續跑
- **Cloud execution**——把任務丟到 OpenAI 雲端跑，你的本機 terminal 空下來

2026 年 1 月 Reddit 一份 500+ 開發者調查顯示 **65% 偏好 Codex CLI、35% Claude Code**。但有趣的是：Claude Code 的 Reddit 討論量是 Codex 的 4x。這個矛盾被 aiengineering.report 稱為「love-hate 張力」——討厭 Claude Code 的人聲音大，但用的人更多。

Benchmark 上雙方互有勝負：

- SWE-bench Verified：Opus 4.7 **87.6%** 領先 GPT-5-Codex 74.9%
- Terminal-Bench 2.0：GPT-5.4 75.1% + GPT-5.3-Codex 77.3% 領先 Opus 4.7 69.4%

誰贏取決於測哪個 benchmark——這個結論本身就很說明問題。

Codex CLI 贏 Claude Code：token 效率（~4x）、啟動速度（Rust 原生）、開源（vs Claude Code 閉源）、cloud execution 原生支援、迭代節奏（每天兩個 release）、Terminal-Bench 分數、ChatGPT Plus 訂閱更划算。

Codex CLI 輸 Claude Code：SWE-bench 分數、長任務 coherence（多步驟 3-4 步後開始失去連貫）、擴展生態（skills、hooks、MCP 都是 Anthropic 的原生優勢）、代碼品質盲測 rework 率。

格局總結：Codex CLI 是 OpenAI 正面對撞 Claude Code 的產品，兩者在 2026 年形成了 CLI agent 賽道的「雙雄格局」。很像 2010 年代的 iOS vs Android——使用體驗、哲學、生態各有擁護者，核心用戶幾乎不會同時全用。「最能打的開發者同時用兩個」這個敘事在 2026 年 Q1 被反覆引用。

4.6 Gemini CLI：慢半拍的巨頭回應

Gemini CLI 是 Google 的補課產品。Claude Code 2024 年內部 → 2025 年 2 月公開；Codex CLI 2025 年 4 月上線；Gemini CLI 2025 年中才追上來。

純 CLI，終端優先，Apache 2.0 開源（google-gemini 組織）。個人 Google 帳號可免費用 Gemini 2.5 Pro 附有量限制，API key / Vertex AI key 付費解鎖更高用量。模型主力是 Gemini 3 Pro（2026 年 Q1 推出）。

Gemini CLI 在 2026 年上半年的每個新功能都能找到對應的 Claude Code 前輩版本：**Plan Mode（2026-02）、Conductor 自動審查（2026-03）、Agent Skills（2026-01 preview）**。完整的 MCP 支援。

但 Gemini CLI 最大的問題不是功能，是 **hallucination**。多個 GitHub issue（#5582、#14754、#13672、#16431）都在抱怨：「Gemini CLI 捏造它自己的 prompt」、「長 context 下忘記 2-3 turn 前的定義」、「幻覺 library 不存在」。一篇標題叫〈The Gemini Hallucination Crisis: How Google Antigravity is Destroying Developer Trust〉的 Medium 文章在 2026 年 3 月爆紅。

Gemini CLI 贏 Claude Code：免費額度、1M context 比 Claude 的 200K（2026-03 前）大、開源。

Gemini CLI 輸 Claude Code：代碼品質（多項 benchmark 落後）、幻覺問題（社群公認最嚴重）、生態成熟度（MCP 追隨 Anthropic、skills 概念抄 Claude Code、plan mode 抄 Claude Code）、品牌信任（Windsurf $2.4B 收購、Gemini 3 hallucination 風波、Antigravity 爭議）。

格局總結：Gemini CLI 在 2026 年 4 月還沒能成為 Claude Code 的第三個對手。它最有存在感的場景是「1M context 真有用的超大檔案處理」或「已經綁在 Google Cloud 生態裡」。對多數開發者，Gemini CLI 是「備用方案」而不是「主力工具」。**Google 需要 Gemini 3 在代碼品質上做出突破性改進，才能真正挑戰 Anthropic/OpenAI 的雙雄格局**。

4.7 次要競品快速掃描

這裡收錄 2026 年 Q2 時其他值得一提的產品，以一段話概括。

Windsurf（Cognition）：2025 年最戲劇性的併購故事。OpenAI 曾同意 $3B 收購，微軟 IP 條款談不攏 2025 年 7 月破局。Google 72 小時內做「reverse acquihire」，$2.4B 挖走 CEO Varun Mohan、共同創辦人 Douglas Chen、核心 R&D 到 DeepMind。剩下的 IP、產品、210 名員工、$82M ARR 被 Cognition AI（Devin 母公司）以約 $250M 收購。目前 Windsurf 是 Cognition 旗下的 IDE 產品，主打 Cascade agent，已失去先機。

Zed：Rust 寫的 120 FPS IDE，不自己做 agent，而是通過 **ACP（Agent Client Protocol）** 整合外部 agent——包含 Gemini CLI、Claude Code、Codex CLI、OpenCode。定位：「最快的編輯器 + 你想要的 agent」。2026 年多個評論把 **Zed + Claude Code** 推為「CLI 玩家的最佳 GUI 組合」。

JetBrains Junie：2026 年 1 月發布，10 個月後整合 Anthropic 官方 Agent SDK。Junie 對 Java/Kotlin 的品質超越 Copilot。定價 $100/年（AI Pro）到 $300/年（AI Ultimate），比 Claude Code/Cursor 的月費都便宜。**Junie 不搶 Claude Code 的用戶，搶的是 Cursor 在 JetBrains 用戶群的市場**。

GitHub Copilot CLI / Agent Mode：2026 年 2 月 25 日正式 GA。Autopilot mode、background delegation、四種特化 agent（Plan/Code Review/Task/Explore）、`.github/agents/` 自定義檔案。天然整合 GitHub（PR、Issue、Action 無縫打通）。**Copilot CLI 是 Claude Code 最大的「機構型」對手**——很多企業已經有 GitHub 合約，加 Copilot 方便合規。但個人開發者口碑普遍不如 Claude Code / Codex CLI。

Warp：不是「AI 編碼工具」，是「AI 原生終端」。它的 Agent Mode 能監控長跑命令、甚至驅動 vim/debugger 這類 TUI。2026 年 2 月推出 Oz（cloud agent orchestration）。**Warp 和 Claude Code 是互補**——很多 Claude Code 用戶在 Warp 裡跑 Claude Code。

Amp（Sourcegraph）：原名 Cody，改名 Amp。**pay-as-you-go 無 markup**——用多少 token 付多少，不加價。主打「team-oriented」功能。相較 Claude Code 的單人訂閱模式，Amp 的團隊協作敘事有差異化，但聲量明顯小一檔。

Devin / Cognition：不是 CLI、不是 IDE，是 SaaS dashboard。你發 Linear/Jira/Slack ticket，Devin 自己去寫程式、跑測試、開 PR。2026 年 2 月更新支援並行 session 和動態 re-planning。定價從 $500/月降到 $20 Core + $2.25/Agent Compute Unit。但 Answer.AI 2025 年研究 20 個真實任務只有 3 個成功。**Devin 和 Claude Code 不直接競爭**——一個是「完全無人」路線，一個是「human + agent」路線。

Replit Agent：Replit Agent 3 可自主跑 200 分鐘。招牌場景是「從 prompt 到部署 URL」——15-40 分鐘內做出可用的 task manager、crypto tracker、帶 Stripe 的 SaaS landing page。**搶的是「非開發者」和「原型階段」用戶，不是 Claude Code 的核心受眾。**

4.8 橫向對比矩陣

維度	Claude Code	Cursor	Cline	Aider	Codex CLI	Gemini CLI
形態	CLI + IDE 整合 + Web + Desktop	獨立 IDE (VS Code fork)	VS Code extension	CLI (Python)	CLI (Rust)	CLI
開源	閉源	閉源	Apache 2.0	Apache 2.0	Apache 2.0	Apache 2.0
模型策略	Anthropic only	多模型 + Composer	BYOK 多模型	BYOK 多模型	OpenAI only	Google only
定價（月）	$20 / $100 / $200	$20 / $60 / $200	免費 + API	免費 + API	免費 + API/Plus	免費額度大
SWE-bench Verified	87.6% (Opus 4.7)	中高	中	中高	74.9% (GPT-5-Codex)	中（幻覺多）
Token 效率	基準	5.5x 更耗	略高於 CC	4.2x 更省	4x 更省	未知

維度	Claude Code	Cursor	Cline	Aider	Codex CLI	Gemini CLI
Day 1 上手	中	最易	易	難	中	中
自主能力	最強	中	弱（要批准）	中弱	最強	中
擴展機制	MCP + Skills + Hooks + Plugins	Extensions + MCP	MCP	無正式 skill	MCP + wire protocol	MCP + skills (preview)
GitHub Stars	112K（客戶端 repo）	閉源	58K	3 萬	75K	未公開
獨特武器	skills/hooks/sub-agent 最成熟	Tab 補全最強	VS Code 原生 + 完全透明	token 效率 + benchmark 透明	Rust 原生 + cloud exec	1M context（2026 Q1 後不再獨特）

4.9 社群口碑的實況快照

三個關鍵統計點，是 2026 年 Q1-Q2 社群最常被引用的：

1. **Vibe Kanban 資料**：Claude Code 使用份額在 2025 年 8 月週限額事件後從 **83% 掉到 70%**。這個事件讓 Anthropic 開始認真思考「限額對用戶體驗的心理傷害」
2. **aiengineering.report 的 Reddit 情緒分析（2026-01）**：500+ 開發者比較裡 65% 偏好 Codex CLI，但 Claude Code 的討論量是 Codex 的 **4x**
3. **Stripe 公開案例**：Stripe 用 Claude Code 在 **4 天**遷移 10,000 行 Scala 到 Java，原估 10 engineering weeks

這三個點拼起來的圖像是：**Claude Code 在「每日 sticky 度」上被挑戰、但在「高強度生產力事件」上依然領先**。前者是訂閱定價和限額政策問題，後者是模型能力和架構成熟度問題。

Anthropic 2026 年 Q1 的戰略顯然在修正前者：4 月的 Opus 4.7 Max Plan 配 Auto mode、1-hour prompt cache、更寬鬆的 weekly quota。這些都是為了讓「重度用戶不再每週末 rage-quit」。

五、橫縱交匯洞察

5.1 歷史如何塑造了今天的位置

回頭看 Claude Code 的崛起，有幾個關鍵的「路徑依賴」決定了它今天的競爭位置：

第一個分叉是選 CLI 而不是 VS Code 擴充（2024 Q4）。如果 Anthropic 當時做 VS Code extension，它會直接撞進 Cursor / Cline 的紅海，第一天就面對「差異化在哪裡」的死亡問題。選 CLI 讓它進入了一個幾乎沒有主流玩家的新品類（Aider 存在但規模太小），得以用一年的時間定義這個品類，而不是搶占一個已經成熟的類別。

這個決策的副作用是——Claude Code 永遠必須克服「新手門檻」。不會用 terminal 的人上不了車，這是為什麼 2025 年 10 月要推出 claude.ai/code、2026 年 4 月要重繪 Desktop App。CLI-first 的定位讓它失去了「非開發者」這個市場，但換來了「重度開發者主力工具」的地位。

第二個分叉是 MCP 協議的開源（2024-11-25）。在一個本來可以做成「Claude 專有」的擴展機制上，Anthropic 選擇把協議開源。這個決策在當時看起來損失很大——等於把自己工具生態的定義權讓給了行業。但事後看這是 Anthropic 2024 年最聰明的一步。

MCP 的開源讓 Anthropic 獲得了兩個巨大的網路效應：一是第三方為所有 MCP client 做的 server 都能被 Claude Code 使用（它等於免費獲得了 Windsurf、Continue、Zed 等所有對手的 server 生態）；二是當 2025 年 OpenAI、Google 也採用 MCP、2025 年 12 月 MCP 被捐贈給 Linux Foundation 時，Anthropic 成為整個協議的「精神主權國」——雖然所有人都在用，但沒人能否認它是 Anthropic 發明的。

第三個分叉是 Max Plan 的訂閱（2025-04-09）。在一個大家還在 pay-per-token 計費的時代，Anthropic 提前推出 $100/$200 的 Max Plan。這個動作在當時看起來像是「在 Claude.ai chat 產品上加訂閱方案」，但實際上它是 Claude Code 未來的經濟引擎。

Cursor 直到 2025 年末才推出 Ultra $200 方案——比 Anthropic 晚了近八個月。這八個月裡，任何一個「重度用 AI 寫程式」的工程師如果想要穩定月費、無限使用量的體驗，Claude Code 是唯一選擇。這個先發優勢鎖定了大量重度用戶，之後很難逆轉。

第四個分叉是 SDK 改名（2025-09-29）。Claude Code SDK → Claude Agent SDK 這個改名，在當時被很多人忽略，覺得只是換名字。但它把 Anthropic 的故事從「做一個 coding agent」改寫成「做一個 agent 平台」。

這個改名的商業含義是——Anthropic 的 North Star 不再是「最好的 coding 工具」，而是「最多的 agent 跑在 Anthropic infra 上」。Claude Code 是這個平台最成熟的 reference implementation，但它只是起點。所以 2026 年我們看到 Anthropic 開始推 Routines、CronCreate、Desktop App——這些都不只是 Claude Code 的功能，它們是 Agent SDK 的 showcase。

5.2 競品的縱向對比：不同起源的不同命運

把主要競品放進時間軸看，它們的起源決定了各自今天的處境：

- **Cursor 從 2023 年開始做 IDE**——一開始就押寶「IDE 是開發者主入口」。所以它的所有設計都圍繞「坐在編輯器前」，用 Tab 做招牌功能。這個路徑讓它在 inline editing 上做到極致，但在「長時間自主任務」這個新品類上永遠慢半拍
- **Cline 從 2024 年做 Claude Dev**——一開始就是「Cursor 的開源替代」，所以它的 DNA 是「in VS Code」。這個路徑讓它在合規和透明度上有先天優勢，但它永遠跳不出 VS Code 這個容器
- **Aider 從 2023 年就做 CLI**——但它的哲學是「Git 是真相」「使用者管 context」，這是工程師文化的極致。這個路徑讓它成為技術圈 icon，但規模化天花板很低
- **Codex CLI 2025 年 4 月才重啟**——起點是 OpenAI 看到 Claude Code 後的「我也要一個」。所以它是純粹的功能 follower，贏在執行力（Rust 重寫、每日兩個 release），但設計哲學沒有差異化
- **Gemini CLI 2025 年中才出**——完全是被動回應，每個功能都能找到 Claude Code 對應版本。加上 Gemini 模型本身的 hallucination 問題，它在 2026 年 Q2 還沒能成為第三個對手

Claude Code 的獨特優勢是：**它的起源不是模仿任何東西**。Boris 2024 年 10 月寫那個「問工程師在聽什麼音樂」的 prototype 時，市場上沒有「CLI-first 的 LLM agent」這個品類。它定義了這個品類。這是任何其他 CLI 競品後來再怎麼做都無法補回的——你可以抄功能，抄不了原型。

5.3 優勢的歷史根源

Claude Code 今天的每個核心優勢，都能追溯到一個具體的歷史決策：

- **擴展生態最成熟**：追溯到 2024-11-25 MCP 開源和 2025-06 Hooks 的推出。Anthropic 很早就決定「Claude Code 的擴展點不在 prompt，在 lifecycle」
- **最強的長任務 coherence**：追溯到 2025-09-29 Sonnet 4.5「30 小時」和 2026-02 Opus 4.6 的 Agent Teams。Anthropic 一直在押「模型能力會消化掉工程問題」，事實證明這個賭贏了
- **最優惠的重度用戶經濟**：追溯到 2025-04-09 Max Plan 和 2025-11-24 Opus 4.5 的 66% 降價。Anthropic 比對手早 8-12 個月鎖定了「Heavy User 友善」的定位
- **最深的 GitHub 整合**：追溯到 2025-05-22 GitHub Actions integration。Anthropic 在 GA 當天就把 CI/CD 打通了，這是後來企業銷售的核心故事

這些優勢不是偶然。它們是一系列早期決策的累積效應——每個決策在當下看都不完美，但合起來形成了難以複製的護城河。

5.4 劣勢的歷史根源

同樣，Claude Code 今天的每個劣勢也都有根源：

- **模型鎖定（只能用 Claude）**：根源是 Anthropic 本身的商業模式。它不可能讓用戶在 Claude Code 裡切換到 GPT-5——那等於自己滅掉自己的 API。這個限制讓 Claude Code 永遠無法像 Cline、Aider 那樣主打「模型自由」
- **閉源**：根源是 Claude Code 包含 Anthropic 的很多 infrastructure 細節（prompt engineering、context 管理策略、安全機制）。開源這些等於把 know-how 給競爭對手。2026-03 源碼洩漏後我們看到 Claude Code 的很多設計被公開研究，但 Anthropic 至今沒有正式開源計劃
- **新手門檻高**：根源是 CLI-first 的起源。不會用 terminal 的人上不了車。這個問題直到 2025-10 的 claude.ai/code 和 2026-04 的 Desktop App 才被部分解決
- **限額政策反彈**：根源是 Max Plan 的「一口價」模式。2025-07 的週限額事件暴露了這個模式的副作用——重度用戶覺得被懲罰。Anthropic 至今還在調整這個平衡

有些當初的「好決策」現在變成了包袱。比如「Claude Code 捆綁 Claude 模型」在 2024-25 年是合理的（Anthropic 的模型夠強），但在 2026 年當 OpenAI Codex CLI 的 GPT-5-Codex 也追上來時，這個綁定變成了 Claude Code 拓寬使用者的阻礙。同樣地，「CLI-first」在 2025 年是獨特武器，但在 2026 年變成了擴大市場的瓶頸。

2026-03-31 的源碼洩漏事件 是這類歷史負債的一次結算。閉源策略在這一天被以最屈辱的方式打破——不是 Anthropic 主動開源，是 `.npmignore` 一行遺漏讓全世界看光。而後的 Claw-code 48 小時 55K stars、墨西哥政府入侵中 75% RCE 是 Claude Code 生成的——這些都是「閉源 + 強自主能力」這個組合積累了兩年的副作用，一夜爆發。

5.5 未來三劇本

基於縱向趨勢和橫向競爭格局，我給 Claude Code 未來 12-18 個月做三個劇本。

最可能的劇本：雙雄格局穩定下來，Claude Code 主導「重度工程師」市場

最可能發生的是：Claude Code 和 Codex CLI 形成穩定的「雙雄」格局（類似 iOS vs Android），兩者在不同用戶群上領先，核心用戶很少同時用。

Claude Code 會繼續主導這些市場：
- 重度企業用戶（Stripe、Shopify 這類大客戶）
- 長任務場景（multi-hour refactor、full-feature implementation）
- 擴展性敏感用戶（自己寫 MCP server、skill、plugin 的 heavy hackers）

Codex CLI 會繼續主導：
- ChatGPT Plus 訂閱者（因為免費、因為已有關係）
- 成本敏感用戶（token 效率 4x 很實在）
- OpenAI 生態用戶（有 GPT-5 其他用例的人）

Cursor 會繼續是第三極，在「Day 1 生產力」和「IDE 粘著度」上站穩，但不再是 Claude Code 的正面威脅——它更多是互補品。

邏輯支撐：過去六個月的數據已經顯示這個趨勢。Claude Code 的 DAU 翻倍、ARR 翻倍、4% commits 佔比都指向「重度用戶池」的穩定擴張。Codex CLI 的 300 萬週活也在持續增長但不蠶食 Claude Code 的核心群體。雙方的 benchmark 勝負互有（SWE-bench vs Terminal-Bench），意味著沒有一邊能通吃。

最危險的劇本：模型能力達到平台期 + 安全事件重創品牌

最危險的情況是兩件事同時發生：

一是 Opus 4.7 之後的幾代模型，能力提升開始收斂——SWE-bench 從 87.6% 推到 92% 需要比前幾次都多的努力，推到 95% 可能根本達不到。如果這個現象發生，Claude Code 最大的護城河（「最強的長任務 coherence」）會被削弱，因為對手也接近相同的天花板。

二是 2026-03 那樣的安全事件再發生一次。源碼洩漏、CVE、被當武器使用——這些都是 Anthropic「safety-first」品牌的最大弱點。如果 2026 年下半年再出一個標誌性的事件（比如一個知名企業因為 Claude Code 的漏洞造成數百萬損失），Anthropic 會失去企業市場的敘事主導權。

合起來，這會讓 Claude Code 從「最強工具」變成「同質化競爭者之一」。Cursor、Codex CLI、甚至 Gemini CLI 都會趁機蠶食市場。ARR 增長會明顯減速，Anthropic 被迫進入「爭奪細分市場」而不是「定義類別」的階段。

邏輯支撐：模型能力的 scaling law 確實存在收益遞減跡象——Opus 4.5 到 4.7 的 SWE-bench 提升（~6.8%）比 Opus 3 到 4 的提升（~30%）小得多。安全事件的頻率也在走向「每季一次」——2025 Q3 的 weekly limit rage、2025 Q4 的 nerfing 爭議、2026 Q1 的 source leak。如果這個曲線延續，2026 下半年可能會有第四個大事件。

最樂觀的劇本：Agent SDK 徹底改變行業結構

最樂觀的情況是，Agent SDK 的網路效應真的起飛——不只是 coding，而是所有 agent 跑在 Anthropic infra 上。Legal assistant、trading bot、SRE automation、customer support、research agent——每個垂直領域都出現一個「標竿 Agent SDK 應用」，它們都使用 Anthropic API 作為主要 provider。

如果這個故事成真，Claude Code 會從「一個產品」升級為「Agent Platform 的 flagship」——就像 iPhone 是 iOS 的 flagship、但真正的生態價值在整個 iOS app store。Anthropic 的商業模式從「賣 token 給寫 coding agent 的人」擴展到「賣 token 給所有 agent」。

這個劇本的關鍵指標是：2026 下半年，有沒有 5+ 個非 coding 領域的「殺手級 Agent SDK 應用」出現？如果有，說明 Anthropic 的 platform play 成功。如果沒有，Agent SDK 還是只是 Claude Code 的底層庫。

邏輯支撐：MCP 已經證明 Anthropic 能推動行業標準，Skills 作為 open standard 也得到了其他平台的採用，Boris Cherny 多次強調「we're building for agents in general」。2026 年 Q1 的 CronCreate、Routines、Dispatch 都是在往「background agent 能跑任何東西」的方向推。這個劇本不是幻想，是 Anthropic 的明牌。

三個劇本裡，我覺得**最可能的劇本概率最高**（60%）、**最危險的次之**（25%）、**最樂觀的最少**（15%）。無論哪個成真，2026-2027 這兩年都會是 AI 編碼工具類別最關鍵的戰場。對開發者來說，現在不是「要不要用 Claude Code」的問題——是「用 Claude Code 來做什麼」的問題。Terminal agent 這個新品類已經贏了，剩下的是看這個品類能擴張到多大。

六、資訊來源

Anthropic 官方

- Anthropic: Introducing the Model Context Protocol (2024-11-25)
- Anthropic: Claude 3.7 Sonnet and Claude Code (2025-02-24)
- Anthropic: Introducing Claude 4 (2025-05-22)
- Anthropic: Code with Claude (2025-05-22)
- Anthropic: Claude Opus 4.1 (2025-08-05)
- Anthropic: Enabling Claude Code to work more autonomously (2025-09-29)
- Anthropic: Introducing Claude Sonnet 4.5 (2025-09-29)
- Anthropic Engineering: Building agents with the Claude Agent SDK
- Anthropic Engineering: Equipping agents for the real world with Agent Skills (2025-10-16)
- Anthropic: Introducing Claude Opus 4.5 (2025-11-24)
- Anthropic: Updates to our consumer terms
- Anthropic: Claude Code on the web (2025-10-20)
- Claude blog: 1M context GA (2026-03-13)
- Claude blog: Redesigning Claude Code on desktop (2026-04-14)
- Claude: Skills 頁面
- Claude Product: Claude Code

官方文檔

- Claude Code Docs: Hooks reference
- Claude Code Docs: Create custom subagents
- Claude Code Docs: Checkpointing
- Claude Code Docs: Scheduled tasks
- Claude Code Docs: Discover plugins
- Claude Code Docs: Skills
- Claude Code Docs: VS Code
- Claude Code Docs: Desktop
- GitHub: anthropics/claude-code releases
- npm: @anthropic-ai/claude-code
- Claude Help Center: Release notes

深度訪談與一手近一手媒體

- Pragmatic Engineer: How Claude Code is built
- Pragmatic Engineer: Building Claude Code with Boris Cherny
- Pragmatic Engineer: DHH's new way of writing code
- Lenny's Newsletter: Head of Claude Code (2026-02-19)
- Developing.dev: Boris Cherny interview
- Station F: Boris Cherny interview
- Every: How to use Claude Code like the people who built it
- Simon Willison: Code with Claude live blog (2025-05-22)
- Simon Willison: Claude Skills (2025-10-16)
- Simon Willison: Claude Code for web (2025-10-20)
- Simon Willison: Claude Opus 4.5 (2025-11-24)
- Sid Bidasaria blog: About

產業媒體

- TechCrunch: Anthropic rolls out $200-per-month Claude subscription (2025-04-09)
- TechCrunch: Anthropic unveils rate limits (2025-07-28)
- TechCrunch: Claude Sonnet 4.5 launch (2025-09-29)
- TechCrunch: Anthropic brings Claude Code to the web (2025-10-20)
- TechCrunch: Cursor $9.9B valuation

- TechCrunch: Google $2.4B acqui-hire Windsurf
- CNBC: Cursor $29.3B funding round
- CNBC: Opus 4.6 vibe working (2026-02-05)
- Bloomberg: $200 monthly Claude subscription
- InfoQ: Anthropic Model Context Protocol spec
- InfoQ: Anthropic Releases Claude Code SDK
- InfoQ: Anthropic Expands Claude Code to Web and Mobile
- VentureBeat: Claude Code 2.1.0
- VentureBeat: Redesigned desktop app and Routines
- MacRumors: Anthropic Rebuilds Claude Code Desktop (2026-04-15)
- SD Times: Anthropic releases Claude 3.7 Sonnet and Claude Code
- Winbuzzer: Claude Code cron scheduling (2026-03)
- Constellation Research: Claude Code revenue doubled since Jan 1
- Tech-Insider: Cursor $60B valuation
- The Next Web: Cursor $50B valuation funding
- Fortune: Anthropic retention rate
- Semafor: Anthropic safety researcher departure

競品分析

- Builder.io: Claude Code vs Cursor
- 56kode: Moving from Cursor to Claude Code
- Cursor forum: Stick with Cursor or switch to Claude Code
- Jock.pl: AI coding harness 2026
- Cline Review 2026
- Cline Series A announcement
- Latent Space podcast with Saoud Rizwan
- Aider official history
- Aider architect mode
- Aider R1+Sonnet SOTA
- OpenAI Codex CLI (official)
- Codex changelog
- InfoQ: Codex CLI Rust rewrite
- SmartScope: Codex vs Claude Code 2026 benchmark

- Google: Introducing Gemini CLI
- Gemini CLI Plan Mode
- Gemini Hallucination Crisis Medium
- Elephas: Windsurf 72 小時拆解
- Zed AI 2026
- JetBrains Junie Review
- GitHub Copilot CLI GA

社群與生態

- @AnthropicAI tweet: Claude 3.7 Sonnet + Claude Code
- @AnthropicAI tweet: Opus 4 + Sonnet 4
- @boris_cherny Threads: Claude Code 2.1.0
- Karpathy: vibe coding 原文
- Karpathy: 80% agent coding (2026-01)
- Simon Willison: tag claude-code
- swyx Latent Space
- obra: Superpowers release
- GitHub: obra/superpowers
- GitHub: forrestchang/andrej-karpathy-skills
- GitHub: affaan-m/everything-claude-code
- GitHub: anthropics/claude-plugins-official
- GitHub: hesreallyhim/awesome-claude-code
- GitHub: VoltAgent/awesome-claude-code-subagents
- Claude Marketplaces
- BuildWithClaude
- Awesome Skills

源碼洩漏與安全事件

- SecurityWeek: Critical vulnerability days after source leak
- Zscaler: Anthropic Claude Code leak
- Check Point Research: CVE-2025-59536
- TheHackerNews: Claude Code leaked
- Layer5: Claude Code source leak GitHub history

- Fortune: Source code leak Mythos
- Cybernews: Claude Code leak spawns fastest GitHub repo
- The Register: Trojanized Claude Code leak
- SOCRadar: Mexican government breach
- TrendMicro: Weaponizing trust
- Straiker: With great agency comes great responsibility

Reddit / 口碑分析

- aiengineering.report: Claude Code vs Codex sentiment analysis
- aitooldiscovery: Claude Code Reddit guide
- Chandler Nguyen: Dropping my $200 Claude Code plan
- ksred: Why I'm back using Cursor
- HN: Claude Code 2.0 discussion
- HN: The Claude Code Leak
- GitHub issue: Weekly Usage Limits
- GitHub issue: Context compaction hang
- GitHub issue: Infinite Compaction Loop
- GitHub issue: Opus regression April 15 2026

商業數據

- SaaStr: Anthropic $14B ARR
- Yahoo Finance: Anthropic $19B ARR
- Anthropic: Series G $380B post-money
- Stormy AI: Claude Code GTM strategy $2.5B ARR

企業案例與黑客松

- Claude: Stripe customer
- Claude: Shopify customer
- Claude Code Hackathon: Cerebral Valley
- Claude Code Hackathon Opus 4.6
- Blockchain.news: Claude Code Hackathon Opus 4.7

Vibe Coding

- Wikipedia: Vibe coding
- Simon Willison: Not all AI-assisted programming is vibe coding
- The New Stack: Vibe coding is passe

架構與源碼分析（原理章節核心引用）

- **本報告「原理」章節基於 2026 年 3 月 31 日意外洩漏的 Claude Code v2.1.88 源碼逆向分析**。分析原件存放於 `~/vault/projects/claude-code-sourcemap_v2.1.88/`（15 章 markdown + 38 張 SVG）
- 原始碼規模：1,884 TypeScript/TSX 檔案、37 頂層目錄
- 參考章節：Ch 00（執行摘要）、Ch 01（啟動效能）、Ch 02（Query Pipeline）、Ch 03（Tool System）、Ch 04（Permission Model）、Ch 05（State Architecture）、Ch 06（Agent Orchestration）、Ch 07（Agent Swarms）、Ch 08（Context Window 管理）、Ch 09（記憶系統）、Ch 10（MCP 整合）、Ch 11（擴展性）、Ch 12（Terminal UI）、Ch 13（未發布功能）、Ch 14（橫切模式）

七、方法論說明

本報告採用**橫縱分析法**（Horizontal-Vertical Analysis）深度研究一個產品的完整樣貌。該方法由數字生命卡茲克提出，融合了語言學中的歷時-共時分析（Saussure）、社會科學中的縱向-橫截面研究設計、商學院案例研究法、以及競爭戰略分析的核心思想。

核心原則：**縱向追時間深度（物件從誕生到當下的完整演變），橫向追同期廣度（當下時間切面上與競品的系統對比），最終在交匯處產出獨到洞察**。

本報告的「原理」章節是本次研究特有的第三個面向——因為 Claude Code 有源碼逆向分析的豐富素材可用，所以除了縱向與橫向，還加入了內部架構的剖析面。這不是橫縱分析法的標準模板，是根據研究對象特性做的延伸。